

In MongoDB, **aggregation** is like a data processing pipeline. Think of it like SQL's `GROUP BY`, `JOIN`, and transformations combined — but more flexible.

👉 You pass your documents through a **pipeline** made up of **stages**. Each stage performs an operation on the documents and passes the result to the next stage.

There are **nine major aggregation pipeline stages** (commonly used). I'll explain each with **examples**, **explanations**, and **real-world use cases**.

🔑 The Nine Aggregation Pipeline Stages

1. MATCH:

with **sample data** and then build our pipeline stage by stage in the **best sequence**.

We'll use a simple `orders` collection as our working dataset:

```
db.orders.insertMany([
  {
    _id: 1,
    customerId: 101,
    items: [
      { product: "Laptop", price: 1200, qty: 1 },
      { product: "Mouse", price: 25, qty: 2 },
    ],
    status: "delivered",
    orderDate: ISODate("2025-01-01"),
  },
  {
    _id: 2,
    customerId: 102,
    items: [{ product: "Keyboard", price: 45, qty: 1 }],
    status: "pending",
    orderDate: ISODate("2025-01-03"),
  },
  {
    _id: 3,
    customerId: 101,
    items: [{ product: "Monitor", price: 300, qty: 1 }],
    status: "delivered",
    orderDate: ISODate("2025-01-05"),
  },
  {
    _id: 4,
    customerId: 103,
    items: [{ product: "Laptop", price: 1200, qty: 2 }],
    status: "delivered",
    orderDate: ISODate("2025-01-06"),
  },
  {
    _id: 5,
    customerId: 104,
    items: [
      { product: "Mouse", price: 25, qty: 1 },
      { product: "Keyboard", price: 45, qty: 1 },
    ],
    status: "cancelled",
    orderDate: ISODate("2025-01-07"),
  },
]);
```

◆ Step 1: \$match (Filtering first — best practice)

Why \$match comes first?

- It **reduces the dataset early** so later stages have fewer documents to process.
- Just like you'd filter rows first in SQL before grouping/sorting.

Example Pipeline:

```
db.orders.aggregate([ { $match: { status: "delivered" } } ] );
```

Result:

```
[
  {
    _id: 1,
    customerId: 101,
    items: [
      { product: "Laptop", price: 1200, qty: 1 },
      { product: "Mouse", price: 25, qty: 2 },
    ],
    status: "delivered",
    orderDate: ISODate("2025-01-01"),
  },
  {
    _id: 3,
    customerId: 101,
    items: [{ product: "Monitor", price: 300, qty: 1 }],
    status: "delivered",
    orderDate: ISODate("2025-01-05"),
  },
  {
    _id: 4,
    customerId: 103,
    items: [{ product: "Laptop", price: 1200, qty: 2 }],
    status: "delivered",
    orderDate: ISODate("2025-01-06"),
  },
];
```

✅ Explanation:

- We only want "delivered" orders since cancelled/pending ones are irrelevant for revenue analysis.
- This trims the collection from **5 docs** → **3 docs** before going forward.

✅ Real-world use case:

- In e-commerce dashboards, analysts usually want **delivered sales only** (not pending/cancelled).
- In healthcare, you'd `$match` patients with "active" status before analyzing treatments.

👉 Next, after `$match`, the logical stage is **`$unwind`** (because `items` is an array and we'll probably want to analyze sales per product).

2. UNWIND

◆ Step 2: `$unwind` (Flattening Arrays)

Why `$unwind` now?

- In our `orders` collection, the `items` field is an **array**.
- If we want to analyze revenue per product, we need each **item inside** `items[]` to become its own document.
- `$unwind` breaks arrays into multiple documents — one document per element.

Example Pipeline:

```
db.orders.aggregate([
  { $match: { status: "delivered" } }, // Step 1: filter
  { $unwind: "$items" }, // Step 2: flatten items array
]);
```

Result:

```
[
  {
    _id: 1,
    customerId: 101,
    items: { product: "Laptop", price: 1200, qty: 1 },
    status: "delivered",
    orderDate: ISODate("2025-01-01"),
  },
  {
    _id: 1,
    customerId: 101,
    items: { product: "Mouse", price: 25, qty: 2 },
    status: "delivered",
    orderDate: ISODate("2025-01-01"),
  },
  {
    _id: 3,
    customerId: 101,
    items: { product: "Monitor", price: 300, qty: 1 },
    status: "delivered",
    orderDate: ISODate("2025-01-05"),
  },
  {
    _id: 4,
    customerId: 103,
    items: { product: "Laptop", price: 1200, qty: 2 },
    status: "delivered",
    orderDate: ISODate("2025-01-06"),
  },
];
```

✓ Explanation:

- Order **#1** originally had 2 items → now split into 2 docs.
- Each `items` entry is now its own object (with `product` , `price` , `qty`).
- This makes it possible to run calculations (like total revenue per product).

✓ Real-world use case:

- In **e-commerce**, one order might contain multiple items. `$unwind` lets you treat them individually (for per-product sales stats).

- In **social media**, you might unwind `tags` array to count tag usage across posts.
- In **healthcare**, you might unwind `prescriptions` array to analyze drug usage per patient.

⚡ Next logical step after `$unwind` is **`$project`**, because now we want to **clean up fields and maybe calculate new values** (like `lineTotal = price × qty`).

Perfect 🙌 — let's move to **Step 3: `$project`** .

3. PROJECT

◆ Step 3: `$project` (Shaping the Data)

Why `$project` now?

- After `$unwind` , each item is on its own line.
- But right now, the structure is still “raw” — we have fields like `items.price` and `items.qty` .
- With `$project` , we can:
 - **Choose only the fields we need**
 - **Rename fields** for clarity
 - **Create computed fields** (e.g., `lineTotal = price × qty`)

Example Pipeline:

```
db.orders.aggregate([
  { $match: { status: "delivered" } }, // Step 1: filter
  { $unwind: "$items" }, // Step 2: flatten array
  {
    $project: {
      _id: 0, // hide MongoDB _id
      customerId: 1,
      product: "$items.product",
      price: "$items.price",
      qty: "$items.qty",
      lineTotal: { $multiply: ["$items.price", "$items.qty"] }, // calculate total
      orderDate: 1,
    },
  },
]);
```

Result:

```
[
  {
    customerId: 101,
    product: "Laptop",
    price: 1200,
    qty: 1,
    lineTotal: 1200,
    orderDate: ISODate("2025-01-01"),
  },
  {
    customerId: 101,
    product: "Mouse",
    price: 25,
    qty: 2,
    lineTotal: 50,
    orderDate: ISODate("2025-01-01"),
  },
  {
    customerId: 101,
    product: "Monitor",
    price: 300,
    qty: 1,
    lineTotal: 300,
    orderDate: ISODate("2025-01-05"),
  },
  {
    customerId: 103,
    product: "Laptop",
    price: 1200,
    qty: 2,
    lineTotal: 2400,
    orderDate: ISODate("2025-01-06"),
  },
];
```

✅ Explanation:

- We've reduced the document to **only the fields we care about** (`customerId` , `product` , `lineTotal`).
- We added a **new computed field** `lineTotal` .
- Now, every line clearly represents **one purchased product** with its total cost.

✅ Real-world use case:

- In **sales reporting**, you almost always need per-item totals.
- In **financial apps**, you'd compute fields like `balance = credits - debits`.
- In **log analysis**, you might project only `timestamp`, `userId`, and a computed `duration`.

⚡ Next logical step after `$project` is **`$group`**, because now we can aggregate — e.g., total spent per customer, or sales per product.

4. GROUP

Excellent 🚀 Let's move into **Step 4: `$group`** — one of the most powerful stages in MongoDB aggregation.

◆ Step 4: `$group` (Aggregating Documents)

Why `$group` now?

- After `$match` → `$unwind` → `$project`, each document represents **one item in an order**, with a `lineTotal`.
- This is the perfect time to **summarize** the data:
 - Total spending per customer
 - Total sales per product
 - Average, min, max, counts

Example Pipeline:

👉 Let's find **total amount spent per customer**.

```

db.orders.aggregate([
  { $match: { status: "delivered" } }, // Step 1: filter
  { $unwind: "$items" }, // Step 2: flatten
  {
    $project: {
      // Step 3: clean & compute
      customerId: 1,
      product: "$items.product",
      lineTotal: { $multiply: ["$items.price", "$items.qty"] },
    },
  },
  {
    $group: {
      // Step 4: aggregate
      _id: "$customerId",
      totalSpent: { $sum: "$lineTotal" }, // sum purchases
      ordersCount: { $sum: 1 }, // count items purchased
      avgOrder: { $avg: "$lineTotal" }, // average per item
    },
  },
]);

```

Result:

```

[
  { _id: 101, totalSpent: 1550, ordersCount: 3, avgOrder: 516.67 },
  { _id: 103, totalSpent: 2400, ordersCount: 1, avgOrder: 2400 },
];

```

✓ Explanation:

- Customer **101** bought 3 items (Laptop , Mouse , Monitor) totaling **\$1550**.
- Customer **103** bought 2 laptops (as 1 line) totaling **\$2400**.
- \$sum , \$avg , \$min , \$max , \$count -like operations are available inside \$group .

✓ Real-world use case:

- **E-commerce:** Total sales per customer, product, or category.
- **Finance:** Total deposits per account.
- **Healthcare:** Average test results per patient.

- **Education:** Average grades per student.

⚡ Now that we've grouped, the **next natural step is \$sort** , so we can rank customers (e.g., who spent the most).

5. SORT:

let's move to **Step 5: \$sort** .

◆ Step 5: \$sort (Ranking & Ordering Results)

Why \$sort now?

- After \$group , we've got summaries per customer.
- To make it useful (like finding **top customers** or **top-selling products**), we need to sort results.
- \$sort works like SQL's ORDER BY .

Example Pipeline:

👉 Let's sort customers by **totalSpent** in descending order (biggest spenders first).

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $unwind: "$items" },
  {
    $project: {
      customerId: 1,
      lineTotal: { $multiply: ["$items.price", "$items.qty"] },
    },
  },
  {
    $group: {
      _id: "$customerId",
      totalSpent: { $sum: "$lineTotal" },
    },
  },
  { $sort: { totalSpent: -1 } }, // Step 5: sort descending
]);
```

Result:

```
[
  { _id: 103, totalSpent: 2400 },
  { _id: 101, totalSpent: 1550 },
];
```

✓ Explanation:

- Customer **103** spent \$2400 (more than 101), so they appear first.
- The `-1` means **descending order**.
- If we used `1`, it would sort ascending (smallest first).

✓ Real-world use case:

- **E-commerce:** Leaderboards for top customers.
- **Streaming:** Most-watched movies/shows.
- **Banking:** Accounts with highest balances.
- **Education:** Rank students by scores.

⚡ After sorting, the next logical steps are:

- `$limit` → to pick only top **N** results.
- `$skip` → to implement pagination.

6. LIMIT

let's move to **Step 6: `$limit`** .

◆ Step 6: `$limit` (Restricting Number of Results)

Why `$limit` now?

- After `$sort` , we often just want the **top N results** (e.g., Top 3 customers).
- `$limit` helps us **restrict the result set** to a fixed number.
- It's super useful for **reports, dashboards, and pagination**.

Example Pipeline:

👉 Let's get the **Top 1 spender** (customer who spent the most).

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $unwind: "$items" },
  {
    $project: {
      customerId: 1,
      lineTotal: { $multiply: ["$items.price", "$items.qty"] },
    },
  },
  {
    $group: {
      _id: "$customerId",
      totalSpent: { $sum: "$lineTotal" },
    },
  },
  { $sort: { totalSpent: -1 } }, // Step 5: sort descending
  { $limit: 1 }, // Step 6: pick only the top 1
]);
```

Result:

```
[{ _id: 103, totalSpent: 2400 }];
```

✅ Explanation:

- `$limit: 1` ensures we only see the **highest spender**.
- If we used `$limit: 3`, we'd get the **Top 3 spenders**.

✅ Real-world use case:

- **E-commerce:** Show "Top 5 products" or "Top 10 customers".
- **Streaming apps:** Show "Top 10 trending movies".
- **Banking:** Show top debtors or savers.
- **Education:** Show top 3 students per exam.

⚡ Now, `$limit` often works together with **`$skip`** to implement pagination.

Perfect Joel 🙌 — let's continue with **Step 7: `$skip`**.

7. SKIP

◆ Step 7: \$skip (Skipping Results)

Why \$skip now?

- \$skip is used to **skip a number of documents** from the top of the result set.
- It's very useful when combined with \$limit for **pagination**.
- Example:
 - Page 1 → skip 0, limit 5
 - Page 2 → skip 5, limit 5
 - Page 3 → skip 10, limit 5

Example Pipeline:

👉 Let's get the **second highest spender** (skip the first one, then show the next).

```
db.orders.aggregate([
  { $match: { status: "delivered" } },
  { $unwind: "$items" },
  {
    $project: {
      customerId: 1,
      lineTotal: { $multiply: ["$items.price", "$items.qty"] }
    }
  },
  {
    $group: {
      _id: "$customerId",
      totalSpent: { $sum: "$lineTotal" }
    }
  },
  { $sort: { totalSpent: -1 } },           // Step 5: sort descending
  { $skip: 1 },                          // Step 7: skip top 1
  { $limit: 1 }                          // then take the next 1
])
```

Result:

```
[  
  { _id: 101, totalSpent: 1550 }  
]
```

✅ Explanation:

- `$skip: 1` removes the first customer (highest spender).
- Then `$limit: 1` ensures we only take **one document after that**.
- Together, they give us the **second highest spender**.

✅ Real-world use case:

- **Pagination:** Show results page by page.
- **Leaderboards:** Show ranks beyond just the top 1.
- **Data exports:** Process large sets in batches.

⚡ Next in our workflow is **Step 8: \$lookup** (joining with another collection).

This is where we can **enrich our customer spend data** with their actual customer details (name, email, etc.) from a `customers` collection.